

Chapter 3: Functions

Part I: Built-in Functions

Outline

- What is a function?
- Built-in Functions
- Function calls
- Parameters and Arguments
- Flow of execution

What is a Function?

- In Python, a function is a named sequence of statements that belong together.

its primary purpose is to help us organize programs into chunks that match how we think about the problem.

- A Function is a “self contained” group of code that accomplish a **specific task**.
- Python has two types of functions, namely **built-in functions**, and **user-defined** functions
- Functions are used to group code into **reusable** blocks.
- Functions help **avoid code duplication**, making code **easier to maintain and debug**.
- In this chapter we focus on built-in functions

Flow of execution

- In order to ensure that a function is defined before its first use, we have to know the order in which statements are executed, which is called the flow of execution.
- Execution always begins at the first statement of the program.
- Statements are executed one at a time, in order from top to bottom.
- Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.
- Function calls are like a detour in the flow of execution.
- Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

Built-in functions

- Built-in functions in Python are pre-defined functions that can be used without additional code.
- Built-in functions come with Python's standard library and don't require any additional imports
- Examples of built-in functions include `print()`, `input()`, `len()`, `str()`, `int()`, `float()`, `list()`, `tuple()`, `dict()`, and `range()`.
- Python's wide range of built-in functions is one of the reasons why it is a popular and versatile programming language.
- These functions perform everyday tasks such as getting user input, printing output, determining the length of objects, converting between data types, and generating sequences.
- The availability of built-in functions simplifies basic tasks and makes Python easy for beginners yet powerful enough for complex projects

Built-in function category

1. Printing and input
print(), input()
2. Type conversion
int(), float(), str(), bool()
3. Mathematical functions
abs(), round(), pow(), max(), min(), sum()
4. Sequence operations
len(), range(), list(), tuple(), dict(), set()
5. String operations
len(), str(), format()
6. Utility functions
type(), id()

Example 1

- Using print(), input() and type()

```
name = input("Enter your name: ")  
print("Hello, " + name)  
print("Type of name variable is:", type(name))
```

- Try it on your IDE and write the interaction

Example 2

- Using `int()`, `float()` and `str()`

```
age = int(input("Enter your age: "))  
height = float(input("Enter your height in meters: "))  
print("You are " + str(age) + " years old and " + str(height) +  
      " meters tall.")
```

- Try it on your IDE and write the interaction

Example 3

- Using abs(), round(), max() and min()

```
number = -4.567
print("Absolute value:", abs(number))
print("Rounded value:", round(number, 2))
numbers = [10, 20, 30, 40]
print("Maximum value:", max(numbers))
print("Minimum value:", min(numbers))
```

- Try it on your IDE and write the interaction

Example 4

- Using len(), list(), tuple(), set() and dict()

```
my_list = [1, 2, 3, 4]
my_tuple = (1, 2, 3, 4, 5)
my_set = {1, 2, 3, 4, 5, 6}
my_dict = {'a': 1, 'b': 2, 'c': 3}
print("Length of my_list:", len(my_list))
print("Length of my_tuple:", len(my_tuple))
print("Length of my_set:", len(my_set))
print("Length of my_dict:", len(my_dict))
```

- Try it on your IDE and write the interaction

Arguments

- An argument is a value we pass into the function as its input when we call the function
- We use arguments so we can direct the function to do different kinds of work when we call it at different times
- We put the arguments in parenthesis after the name of the function

Modules

- The built-in functions mentioned so far are part of Python's standard library and do not require any additional packages to be imported.
- however, as you progress in Python, you might find yourself needing additional functionality that isn't covered by the built-in functions.
- In such cases, you can import various packages and modules.
- A few commonly used ones:
- math, random, datetime, os, sys
- To use these modules apply import statement

Modules

- **math**: for advanced mathematical functions beyond the basic arithmetic provided by the built-in functions.
`import math`
- **random**: to generate random numbers and performing random operations
`import random`
- **datetime**: for working with dates and times
`import datetime`
- **os**: to interact with the operating system, such as reading or writing to the filesystem
`import os`
- **sys**: to access system-specific parameters and functions.

```
#Check the output
# 1. Using math package
import math
print(math.sqrt(16))  # Square root
print(math.pi)       # Value of pi
print(math.sin(math.radians(90))) # Sine of 90 degrees

# 2. Using random package
import random
print('Random integer between 1 and 100:')
print(random.randint(1, 100))
print('Random choice from a list: ')
print(random.choice(['Bin', 'Alex', 'Ajwa', 'Abel', 'Alazar']))
```

```
# 3. Using datetime
import datetime
now = datetime.datetime.now()
print("Current date and time:", now)
print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)
```

Check it on IDE

```
# 4. Using os package
import os

print("Current working directory:", os.getcwd())
print("List of files and directories:", os.listdir('.'))

# 5. Using sys package
import sys

print("Python version:", sys.version)
print("Command line arguments:", sys.argv)
```

Try it on your IDE

```
# 5. Using sys package
import sys

print("Python version:", sys.version)
print("Command line arguments:", sys.argv)
```

Selected functions in math module

Function	Returns
<code>sqrt(x)</code>	The square root of x . ($x \geq 0$)
<code>trunc(x)</code>	Truncates floating-point value x to an integer.
<code>cos(x)</code>	The cosine of x in radians.
<code>sin(x)</code>	The sine of x in radians.
<code>tan(x)</code>	The tangent of x in radians.
<code>exp(x)</code>	e^x
<code>degrees(x)</code>	Convert x radians to degrees (i.e., returns $x \cdot 180/\pi$)
<code>radians(x)</code>	Convert x degrees to radians (i.e., returns $x \cdot \pi/180$)
<code>log(x)</code> <code>log(x, base)</code>	The natural logarithm of x (to base e) or the logarithm of x to the given <i>base</i> .

Different way to import the module math

- There are several ways of importing the module math with different usage:

1. Import the entire module:

```
import math
```

Import the entire module with alias:

```
import math as m
```

You will need to prefix **math.** Or **m.** to access its functions

```
import math
print(math.sqrt(16)) # Output: 4.0
print(math.pi)      # Output: 3.141592653589793
```

- Takeaway exercise: try it with import math as m

Different way to import the module math cont'd

2. Importing Specific Functions from the Module

- this method imports specific functions or variables from the math module
 - from math import sqrt, sin, cos, pi, exp, log
- Does not need to prefix them with `math`

```
from math import sqrt, pi
print(sqrt(16))    # Output: 4.0
print(pi)          # Output: 3.141592653589793
```

Different way to import the module math cont'd

3. Importing All Functions from the Module

- This method imports all functions and variables from the `math` module

```
from math import *  
  
print(sqrt(16))    # Output: 4.0  
print(pi)          # Output: 3.141592653589793  
print(sin(radians(90)))#1,0  
print(exp(1))# 2,718281828459045  
print(log(100)) #4.605170185988092  
print(exp(4.605170185988092))# 100.000000000000004
```

Takeaway Exercise

1. Write a program that calculates the area and circumference of a circle whose radius is entered from the keyboard.
 - Use the built-in value of pi
 - Make sure the value of radius is greater than zero
 - Make sure the value entered for radius is numeric
 - Print the result of the computation
2. Write a program to calculate the sin, cos, tan of an angle received from the user
 - Decide whether to enter in degrees or radians
 - Make the necessary type conversion
3. Write a program that generates a random number between 1 and 1000
4. Write a program that accepts two integers and prints the greatest common divisor (GCD) and the least common multiple (LCM).

End of chapter 3

Chapter 5

Function part II (User defined functions)

Outline

- User-Defined Functions
- Fruitful and void functions
- Return vs Print
- Function Scope
- Decomposition and Abstractions

Function recap

- A function packages a computation consisting of multiple steps into a form that can be easily understood and reused.
- Earlier we have used predefined functions
- Here, you will learn how to design and implement your own functions
- Using the process of stepwise refinement, you will be able to break up complex tasks into sets of cooperating functions.
- Two types: built-in and user defined

Function recap

- A function is a sequence of instructions with a name.
- You have already encountered several functions
- example, the built-in function, round contains instructions to round a floating-point value to a specified number of decimal places.
- You call a function in order to execute its instruction
 - `price = round(15.27945, 2)` # Sets result to 15.28
 - By using the expression `round(15.27945, 2)`, your program calls the round function, asking it to round 15.27945 to two decimal digits.
 - instructions of the round function execute and compute the result.
 - The round function returns its result back to where the function was called and your program resumes execution
 - The “input” values on which the called function works are the **arguments** of the function
 - The “output” that the round function computes is called the **return value**

Function recap

- Functions can receive multiple arguments, but they **return only one value**.
- It is also possible to have functions with no arguments.
- An example is the random function that requires no argument to produce a random number.
- the return value of a function is returned to the point in your program where the function was called.
- It is then processed according to the statement containing the function call
- as a user of the function, you don't need to know how the function is implemented.
 - You just need to know the specification of the function:
 - If you provide arguments x and n, the function returns x rounded to n decimal digits. Engineers use the term black box for a device with a given specification but unknown implementation

Takeaway Exercises

1. Consider the function call `round(3.14159, 2)`. What are the arguments and return values?
2. What is the return value of the function call `round(round(4.499, 2), 0)`?
3. What is the return value of `abs(-4.0)`?
4. It is possible to determine the answer to Q3 without knowing how the `abs()` function is implemented. Use an engineering term to describe this aspect of the `abs()` function.

User defined function

- When you design your own functions, you will want to make them appear as black boxes to other programmers.
- Those programmers want to use your functions without knowing what goes on inside.
- Even if you are the only person working on a program, making each function into a black box pays off: there are fewer details that you need to keep in mind. What are they?

User defined function

- Define a function that calculates volume of a cube
 - Pick a name for the function (**cubeVolume**).
 - Define a variable for each argument (sideLength).
 - These variables are called the parameter variables.
 - Put all this information together along with the **def** reserved word to form the first line of the function's definition:
def cubeVolume(sideLength) : # This line is called the header of the function.
 - Next, specify the body of the function. The body contains the statements that are executed when the function is called.
 - The volume of a cube of side length sideLength is **sideLength ** 3**.
 - We will store this value in a variable called **volume**:
volume = sideLength ** 3
 - In order to return the result of the function, use the return statement:
return volume

Define a function

- If you run a program containing just the function definition, then nothing happens.
- In order to test the function, your program should contain
 - The definition of the function
 - Statements that call the function and print the result.

#defining the function that accepts the side of a cube and returns the volume

```
def cubeVolume(sideLength):
```

```
    volume = sideLength ** 3
```

```
    return volume
```

```
result1 = cubeVolume(3) #call the function with argument 3 and store the result in result1
```

```
result2 = cubeVolume(9) # call the function with argument 9 and store the result in result2
```

```
print("A cube with side length 3 has volume", result1)
```

```
print("A cube with side length 9 has volume", result2)
```

Function Definition

Syntax `def` *functionName*(*parameterName*₁, *parameterName*₂, . . .) :
 statements

The diagram shows a Python function definition with the following code:

```
def cubeVolume(sideLength) :  
    volume = sideLength ** 3  
    return volume
```

Annotations with arrows pointing to the code:

- Function header** points to the `def` keyword.
- Name of function** points to `cubeVolume`.
- Name of parameter variable** points to `sideLength`.
- Function body, executed when function is called.** points to the indented lines `volume = sideLength ** 3` and `return volume`.

return statement
exits function and
returns result.

Calling a function

- it is important that you define each function before you call it
 - Unless the function is defined in another function (next slide)
- Calling a function before it is defined raises a compile time error (NameError)

```
print(cubeVolume(10)) #call
def cubeVolume(sideLength): #define
    volume = sideLength ** 3
    return volume
```

NameError: name 'cubeVolume' is not defined

- The compiler does not know that the cubeVolume function will be defined **later** in the program.

Calling a function

- Note that a function can be called from within another function before it has been defined. For example, the following is perfectly legal:

```
def main():  
    result = cubeVolume(10)  
    print("A cube with side length 10 has volume", result)  
def cubeVolume(sideLength):  
    volume = sideLength ** 3  
    return volume  
main()
```

Example (Check the output)

```
def power(base, exponent):  
    val = base  
    for x in range(1, exponent):  
        print(x, val)  
        val = val * base  
    print(exponent, val)  
  
#Example call the power function  
power(2, 9) # what is the output?
```

Example cont'd (check the output)

```
def divide_num(num1, num2):  
    try:  
        if num2 == 0:  
            print("Division by zero is not allowed.")  
        else:  
            result = num1 / num2  
            print(f"The result is {result}")  
    except TypeError:  
        print("Data type error, both arguments must be numbers.")  
  
# Example call the funtion passing two arguments  
divide_num(100, 2)    # The result is 50.0  
divide_num(100, 0)    # Division by zero is not allowed.  
divide_num(100, "num2") # Data type error, both arguments must be numbers.  
divide_num(100, 'A')  # Data type error, both arguments must be numbers.
```

Takeaway Exercises

1. Provide an alternate implementation of the body of the `cubeVolume` function that does not use the exponent operator.
2. Define a function `squareArea` that computes the area of a square of a given side length.
3. Consider this function:

```
def mystery(x, y) :  
    result = (x + y) / (y - x)  
    return result
```

What is the result of the call `mystery(2, 3)`?

Takeaway Exercises

- What is the output of the following program?

```
def main() :  
    a = 4  
    print(mystery(a + 1))  
def mystery(x) :  
    y = x * x  
    return y  
main()  
|
```

Takeaway Exercises

- What is the output of the following program?

```
def main() :  
    a = 5  
    print(mystery(a))  
def mystery(n) :  
    n = n + 1  
    n = n + 1  
    return n  
main()
```

Takeaway Exercises

#Q1 What is the output?

```
def main() :  
    a = 4  
    print(mystery(a + 1))
```

```
def mystery(x) :  
    y = x * x  
    return y  
main()
```

Q2 What is the output?

```
def main() :  
    a = 5  
    print(mystery(a))
```

```
def mystery(n) :  
    n = n + 1  
    n = n + 1  
    return n  
main()
```

Parameter Passing

- parameter passing allows functions to receive inputs, process them, and return outputs.
- Python supports several ways of passing parameters to functions, providing flexibility and control over how data is handled.
 - Positional parameter – passed by position
 - Keyword parameter – passed by name
 - Default parameter – provide default values
 - Arbitrary Positional (*args) - Collect extra positional arguments
 - Arbitrary keyword(**args) - Collect extra keyword arguments

Passing parameters cont'd

- Positional parameters are the most basic type of parameters. They are passed to the function in the order in which they are defined

```
def greet(name, message):  
    print(f"Hello, {name}! {message}")
```

```
# Calling the function with positional arguments  
greet("Selamawit", "Good morning!") # Output: Hello, Selamawit! Good morning!
```

Passing parameters cont'd

- Keyword parameters allow you to pass arguments to a function using the parameter names,
 - makes the function calls more readable and allowing you to skip the order.

```
def greet(name, message):  
    print(f"Hello, {name}! {message}")  
  
# Calling the function with keyword arguments  
greet(message="Good evening!", name="Selamawit") # Output: Hello, Selamawit! Good evening!
```

Passing parameters cont'd

- Default parameters allow you to define default values for parameters.
- If an argument for that parameter is not provided, the default value is used.

```
def greet(name, message="Hello!"):  
    print(f"{message} {name}")  
  
# Calling the function without the message parameter  
greet("Abel")    # Output: Hello! Abel  
  
# Calling the function with both parameters  
greet("Abel", "Good night!")    # Output: Good night! Abel
```

Passing parameters cont'd

- Sometimes you may need to pass a variable number of positional arguments to a function. You can achieve this using `*args`
 - `*args` collects extra positional arguments as a tuple

```
def sum_numbers(*args):  
    return sum(args)  
  
# Calling the function with a variable number of arguments  
print(sum_numbers(1, 2, 3))           # Output: 6  
print(sum_numbers(4, 5, 6, 7, 8))     # Output: 30  
|
```

Passing parameters cont'd

- you can also pass a variable number of keyword arguments using `**kwargs`
 - `**kwargs` collects extra keyword arguments as a dictionary

```
def print_details(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
# Calling the function with a variable number of keyword arguments  
print_details(name="Selamawit", age=30, city="Addis Ababa")  
# Output:  
# name: Selamawit  
# age: 30  
# city: Addis Ababa
```

Combining different parameter types

- You can combine different types of parameters in a single function. The order should be: standard positional parameters `*args`, keyword parameters, and `**kwargs`

```
def example_function(a, b=2, *args, **kwargs):  
    print(f"a: {a}, b: {b}")  
    print("args:", args)  
    print("kwargs:", kwargs)  
  
# Calling the function with various types of arguments  
example_function(1, 3, 4, 5, x=6, y=7)  
# Output:  
# a: 1, b: 3  
# args: (4, 5)  
# kwargs: {'x': 6, 'y': 7}
```

Passing parameters cont'd

- Python uses a mechanism known as "pass-by-object-reference" or "pass-by-assignment".
- This means mutable objects such as lists, and dictionaries can be modified within the function, while immutable objects such as integers, strings cannot.

Passing parameters cont'd

- Python uses "pass-by-object-reference".
- "Pass-by-object-reference" (or "pass-by-assignment") is a term used to describe how Python handles variable references passed to functions.
 - Note that this can be a bit tricky, especially for those coming from languages that distinguish clearly between pass-by-value and pass-by-reference

What does it mean by 'pass by object reference'?

- In Python, variables are references to objects.
 - When you pass a variable to a function, you are passing a reference to the object the variable refers to, not the actual object itself.
 - whether you can modify the object inside the function depends on the mutability of the object.

Mutable vs Immutable Objects

- **Mutable objects:** are objects that can be changed or modified in place.
 - Examples include **lists**, and **dictionaries**
- **Immutable objects:** are objects that cannot be changed or modified in place.
 - Examples include **integers**, **floats**, **strings**, and **tuples**.

How python passes immutable objects

- For immutable objects like **integers** and **strings**, since the objects themselves cannot be changed, any modification within the function results in the creation of a new object.
- The original reference remains unchanged.
- Example:

```
def modify_number(n) :  
    n += 1  
    print(f"Inside function: n = {n}")  
  
num = 10  
modify_number(num)  
print(f"Outside function: num = {num}")
```

Immutable example cont'd

```
def modify_string(s):  
    s += " world"  
    print(f"Inside function: s = {s}")  
  
my_string = "Hello"  
modify_string(my_string)  
print(f"Outside function: my_string = {my_string}")
```

Immutable example cont'd

- In examples (previous slide) the original variable `num` and `my_string` remain unchanged outside the function because integers and strings are immutable.
- The operations inside the function create new objects rather than modifying the existing one

```
def modify_number(n):  
    n += 1  
    print(f"Inside function: n = {n}")  
  
num = 10  
modify_number(num)  
print(f"Outside function: num = {num}")
```

Mutable Objects (Lists and Dictionaries)

- For mutable objects, changes made within the function affect the original object because the reference points to the same memory location.
- Example List (Check it for yourself), what is the output? What would happen if you call the function again and again?

```
def modify_list(lst):  
    lst.append(4)  
    print(f"Inside function: lst = {lst}")  
  
my_list = [1, 2, 3]  
modify_list(my_list)  
print(f"Outside function: my_list = {my_list}")
```

Mutable Objects (Lists and Dictionaries)

- Example List cont'd (Check it for yourself) What is the output?

```
def modify_list(lst):  
    lst.append('apple')  
    print(f"Inside function: lst = {lst}")  
  
my_list = ['cherry', 'orange', 'lemon']  
modify_list(my_list)  
print(f"Outside function: my_list = {my_list}")
```

Mutable Objects (Lists and Dictionaries) cont'd

- Example with dictionary
- Let's define a function (next slide) that modifies a dictionary by
 - adding a new key-value pair,
 - Removing a new key-value pair, and
 - observe how changes made inside the function affect the dictionary outside the function

```
def add_student(student_dict, student_id, student_name):  
    #Add a student to the dictionary  
    student_dict[student_id] = student_name  
    print(f"Inside function: {student_dict}")  
  
def remove_student(student_dict, student_id):  
    #Remove a student from the dictionary  
    if student_id in student_dict:  
        del student_dict[student_id]  
        print(f"Inside function after removal: {student_dict}")  
    else:  
        print(f"Student ID {student_id} not found.")  
  
# Initial student dictionary  
students = {  
    'UGR/1203/24': "Ruth",  
    'UGR/1001/24': "Abay",  
    'UGR/1325/24': "Abel",  
}  
  
# Adding a new student  
print("Before adding a new student:", students)  
add_student(students, 'UGR/1526/24', "David")  
print("After adding a new student:", students)  
  
# Removing a student  
print("\nBefore removing a student:", students)  
remove_student(students, 'UGR/1001/24')  
print("After removing a student:", students)
```

Example mutable object (try it)

```
def modify_list(lst):  
    lst.append(4)  
  
my_list = [1, 2, 3]  
modify_list(my_list)  
print(my_list)    # Output: [1, 2, 3, 4]
```

Example immutable object (try it)

```
def modify_string(str1):  
    str1 += " world"  
  
my_string = "Hello"  
modify_string(my_string)  
print(my_string)    # Output: Hello
```

Fruitful and void functions

- Fruitful functions are functions that return a value
 - such functions use the return statement to send a value back to the caller.
 - value returned can be of any data type, such as numbers, strings, lists, dictionaries, or even another function

```
def add_numbers(a, b):  
    return a + b  
  
# Calling the fruitful function  
result = add_numbers(3, 5)  
print(result)    # Output: 8
```

Void functions

- Void functions, are functions that do not return a value.
 - Instead of using the return statement to send a value back to the caller, void functions typically perform some action like printing output, modifying a global variable, or changing the state of an object

```
|  
def greet(name):  
    print(f"Hello, {name}!")  
  
# Calling the void function  
greet("Abenezer")    # Output: Hello, Abenezer!
```

Combined fruitful and void functions

- we can use both fruitful and void functions (Check the output)

```
# Fruitful function
def square(number):
    return number * number

# Void function
def print_square(number):
    result = square(number)    # Calling the fruitful function
    print(f"The square of {number} is {result}")

# Calling the void function
print_square(4)    # Output: The square of 4 is 16
|
```

Takeaway Exercise

1. Write a program that defines a function to collect scores, and a function to modify scores. The main program calls the `read_score` function to collect the scores and the `modify_score` function to change the scores. Finally the main program displays the scores of the students before and after the modification is applied. Hint:
 - Function `read_score`:
 - repeatedly prompts the user to enter a score (make sure data entry error is handled with `try/except`)
 - Adds each valid score to the 'scores' list, which was initially empty
 - Stops when the user types 'done'
 - Returns the list of scores
 - Function `modify_score`:
 - Takes a list of scores as an argument
 - Modifies each score in the list (e.g., by adding 5 points to each score).
 - Prints the modified list
 - This function is void because it does not return a value. Instead, it modifies the list in place
 - Main program
 - Collects scores using 'read_score'
 - Prints the original scores
 - Passes the list of scores to 'modify_score'
 - Prints the modified scores

```

def read_score():
    """Collects multiple scores from the user until they type 'done'."""
    scores = []
    while True:
        input_str = input("Enter a score (or type 'done' to finish): ")
        if input_str.lower() == 'done':
            break
        try:
            score = int(input_str)
            scores.append(score)
        except ValueError:
            print("Please enter a valid integer score.")
    return scores

def modify_score(scores):
    """A void function that modifies the scores list."""
    for i in range(len(scores)):
        scores[i] += 5 # Example modification: adding 5 points to each score
    print("Scores after modification:", scores)

# Collect scores from the user
original_scores_list = read_score()
print("Original scores:", original_scores_list)

# Modify the scores using a void function
modify_score(original_scores_list)

# Display the modified scores
print("Original scores affected by modify_score:", original_scores_list)

```

Scope

- **Scope** refers to the **visibility** and **lifetime** of variables within a program. In Python, function scope determines where a variable can be accessed or modified.
 - **Local Scope**: Variables declared inside a function are local to that function and can only be accessed within that function.
 - **Global Scope**: Variables declared outside of all functions are global and can be accessed from any part of the program.
 - **Nonlocal Scope**: Variables declared inside nested functions. They are neither local to the inner function nor global to the outer function but fall somewhere in between

Example (try it)

```
num1 = 10    # Global variable

def outer_function():
    num2 = 15    # Local variable to outer_function
    def inner_function():
        nonlocal num2    # Refers to num2 in the enclosing function
        num2 += 2
        print(num2)
    inner_function()
    print(num2) #refers to the local num2

outer_function()
print(num1)
```

Function decomposition and abstraction

- **Decomposition** is the process of breaking down a complex problem or system into smaller, more manageable parts.
- In programming, it means dividing a program into functions, each responsible for a specific task.
- Why decomposition?
 - Simplifies the program structure.
 - Enhances code readability and maintainability.
 - Facilitates debugging and testing.

Example decompose task into input, process and output:

```
def input_data():  
    # Code to read data  
    pass  
  
def process_data(data):  
    # Code to process data  
    pass  
  
def output_data(data):  
    # Code to write data  
    pass  
  
def main():  
    data = input_data()  
    processed_data = process_data(data)  
    output_data(processed_data)  
  
main()
```

Function Abstraction

- **Abstraction** refers to the concept of hiding the complex implementation details of a function and exposing only the necessary parts to the user.
- It allows you to use functions without needing to understand their inner workings
- Why abstraction?
 - Reduces complexity.
 - Promotes code reusability.
 - Enhances focus on high-level logic.

Example

```
def calculate_area(radius):  
    #Calculate the area of a circle given the radius  
    from math import pi  
    return pi * radius ** 2  
  
# Using the function without knowing its internal implementation  
area = calculate_area(7)  
print(round(area,0))
```

Takeaway Exercises

- Write a simple program that define a function to accept number (s) from the user and another that
 1. Computes the larger of two integers
 2. computes the smallest of three floating-point numbers
 3. Checks whether an integer is a prime number, returning True if it is and False otherwise
 4. Prints the integers in the reversed order
- Test your functions in a main program
 - Make sure the main program calls the functions with appropriate arguments

Recursive function

- A recursive function is a function that calls itself in order to solve a problem.
- Recursive functions are useful for solving problems that can be broken down into smaller, simpler subproblems of the same type.
- They typically involve a base case and a recursive case:
- **Base Case:** The simplest instance of the problem, which can be solved directly without recursion.
 - The condition under which the function returns a result without making a recursive call.
 - crucial to prevent infinite recursion and provide a stopping point.
- **Recursive Case:** The part of the function where the function calls itself with a modified argument, working towards reaching the base case.

Example: factorial

- The factorial of a non-negative integer n (denoted mathematically $n!$) is the product of all positive integers less than or equal n
- The recursive definition of a factorial is

$$0! = 1 \text{ (base case)}$$

$$n! = n \times (n - 1)! \text{ for } n > 0 \text{ (recursive case)}$$

Example factorial cont'd

```
#Recursive implementation of factorial
def factorial(n):
    if n == 0:
        return 1    # Base case
    else:
        return n * factorial(n - 1)    # Recursive case

# Example usage
print(factorial(5))    # Output: 120
```

Example: the Fibonacci sequence

- The Fibonacci sequence can also be defined recursively. The sequence starts with 0 and 1, and each subsequent number is the sum of the previous two, mathematically represented as:

$$F(0) = 0 \text{ (base case)}$$

$$F(1) = 1 \text{ (base case)}$$

$$F(n) = F(n - 1) + F(n - 2) \text{ for } n > 1 \text{ (recursive case)}$$

Example implementation: the Fibonacci sequence

```
#Recursive implementation of Fibonacci
def fibonacci(n):
    if n <= 0:
        return 0    # Base case
    elif n == 1:
        return 1    # Base case
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)    # Recursive case

# Example usage
print(fibonacci(5))    # Output: 5
```

```
#Return the nth Fibonacci number using recursion.
def fibonacci_recursive(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

# Example usage
n = 10
fib_sequence = [fibonacci_recursive(i) for i in range(n)]
print(f"First {n} Fibonacci numbers (recursive): {fib_sequence}")
```

Advantages of recursive function

- **Simpler Code:** For problems that naturally fit a recursive pattern such as the following, recursive solutions can be more intuitive and shorter
 - tree traversals,
 - factorial calculation,
 - Fibonacci sequence, etc.),
- **Problem Decomposition:** Recursive functions help break down complex problems into simpler sub-problems, making them easier to understand and solve.

Disadvantages of recursive functions

- **Performance:** Recursive functions can be less efficient and slower than iterative solutions due to
 - the overhead of multiple function calls and
 - potential stack overflow for deep recursions.
- **Stack Overflow:** Excessive recursion depth can lead to stack overflow errors if
 - the base case is not reached quickly enough or if
 - the recursion is too deep.

Iterative approach

- The iterative approach uses a loop to generate the Fibonacci sequence
- Ex. Compare and contrast iterative and recursive functions

Iterative implementation of Fibonacci

```
#Generate the first n Fibonacci numbers using iteration.
def fibonacci_iterative(n):
    fib_sequence = []
    a, b = 0, 1
    for _ in range(n):
        fib_sequence.append(a)
        a, b = b, a + b
    return fib_sequence

# Example usage
n = 10
print(f"First {n} Fibonacci numbers (iterative): {fibonacci_iterative(n)}")
```

Takeaway Exercise

1. Define a function that computes the factorial of an integer received from the user using iterative approach
 - Make sure data entry error is handled using try/except block
 - Test your function with appropriate input and output interaction
2. Discuss application areas of Fibonacci series in the real world

End of chapter 3&5